

Data Structures and Algorithms

Lab 06 - Stack and Its Applications

Instructor: Sidrat-ul-Muntha

READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. **WORKING** – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).
 2. **DEADLINE** – One Week after assigned this.
 3. **SUBMISSION** – This assignment needs to be submitted in a soft copy.
 4. **WHERE TO SUBMIT** – Please visit your LMS.
 5. **HOW TO SUBMIT** – Rename all .java files according to the task number (e.g., Task1.java, Task2.java). Place all required .java files in a .zip file named 000-00-0000_assignment.zip and submit the ZIP file on the LMS.
-

Learning Objectives

By the end of this lab, students will be able to:

- Implement a Stack using arrays and linked lists.
 - Perform basic Stack operations such as push, pop, peek, and traversal.
 - Apply Stack data structures to solve problems such as string reversal, palindrome checking, and balanced parentheses.
 - Implement real-world applications of Stacks using multiple Stack structures.
 - Apply Stacks to implement browser back/forward navigation and undo/redo functionality.
-

Task 1: Stack Using Array

Create a class named Stack that implements a stack using an array.

The Stack should follow the LIFO (Last-In, First-Out) principle.

Implement the Following Methods

- **push(data)** – Add a new element to the top of the stack.
- **pop()** – Remove and return the top element from the stack.
- **peek()** – Return the top element without removing it.
- **isEmpty()** – Check whether the stack is empty.
- **isFull()** – Check whether the stack is full.
- **display()** – Display all elements of the stack from top to bottom.

Demonstration

In the **main()** function, create a Stack and demonstrate all the above operations.

Task 2: Stack Using Linked List

Create a class named Stack that implements a stack using a singly linked list.

Each node should contain:

- data
- next

The top pointer should point to the first node of the linked list.

Implement the Following Methods

- **push(data)** – Add a new node to the top of the stack.
- **pop()** – Remove and return the top node.
- **peek()** – Return the value at the top without removing it.
- **isEmpty()** – Check whether the stack is empty.
- **display()** – Display all elements from top to bottom.

Demonstration

In the **main()** function, create a Stack using a linked list and demonstrate all the above operations.

Task 3: Reverse a String Using Stack

Implement a **stack of characters** and use it to reverse a given string.

Requirements

- Push each character of the string onto the stack.
- Pop the characters one by one to construct the reversed string.

- Do not use a built-in function to directly reverse the string.

Example

Input:

DATA

Output:

ATAD

Demonstration

In the **main()** function, take a string as input and demonstrate the string reversal.

Task 4: Palindrome Using Stack

Implement a method that uses a Stack to check whether a given string is a palindrome.

A palindrome reads the same forward and backward.

Example

Input:

MADAM

Output:

true

Input:

HELLO

Output:

false

Demonstration

In the **main()** function, call the method with different strings and display the results.

Task 5: Balanced Parentheses Using Stack

Write a method that uses a **Stack** to check whether a given expression contains balanced parentheses.

The expression may contain:

- ()
- {}
- []

Example

Input:

{[()]}

Output:

Balanced

Input:

{[(())]}

Output:

Not Balanced

Requirements

- Push opening brackets onto the stack.
- When a closing bracket is encountered, check whether it matches the top opening bracket.
- The expression is balanced only if all brackets are correctly matched.

Demonstration

In the **main()** function, test the method with multiple expressions.

Task 6: Browser Back and Forward History

Create a simple **browser history system** using Stacks.

The system should maintain two stacks:

- **Back Stack** – stores previously visited pages.
- **Forward Stack** – stores pages that can be visited again after going back.

Implement the Following Methods

- **visitPage(url)** – Visit a new page and add the current page to the Back Stack.
- **goBack()** – Move the current page to the Forward Stack and display the previous page.
- **goForward()** – Move the current page to the Back Stack and display the next page.
- **currentPage()** – Display the currently active page.

Example

Start with:

- Back Stack: []
- Current Page: Home
- Forward Stack: []

1. Visit Google

Operation:

- visitPage("Google")

Before visiting Google, Home is pushed into the Back Stack.

- Back Stack: [Home]
- Current Page: Google
- Forward Stack: []

2. Visit YouTube

- visitPage("YouTube")

Google is pushed into the Back Stack.

- Back Stack: [Home, Google]
- Current Page: YouTube
- Forward Stack: []

3. Visit GitHub

- visitPage("GitHub")
- Back Stack: [Home, Google, YouTube]
- Current Page: GitHub
- Forward Stack: []

4. Go Back

- goBack()

The current page GitHub goes into the Forward Stack.

The top of the Back Stack (YouTube) becomes the current page.

- Back Stack: [Home, Google]
- Current Page: YouTube
- Forward Stack: [GitHub]

5. Go Back Again

- goBack()

- Back Stack: [Home]
- Current Page: Google
- Forward Stack: [GitHub, YouTube]

6. Go Forward

- goForward()

The current page Google goes into the Back Stack.

YouTube is removed from the Forward Stack and becomes current.

- Back Stack: [Home, Google]
- Current Page: YouTube
- Forward Stack: [GitHub]

7. Visit Facebook

Now suppose we visit a **new page**:

- visitPage("Facebook")

YouTube goes into the Back Stack, and importantly, the **Forward Stack is cleared** because we have started a new browsing path.

- Back Stack: [Home, Google, YouTube]
- Current Page: Facebook
- Forward Stack: []

Task 7: Undo and Redo Functionality

Create a simple **Undo/Redo system** using two Stacks.

The system should maintain:

- **Undo Stack** – stores previous actions.
- **Redo Stack** – stores actions that have been undone.

Implement the Following Methods

- **performAction(action)** – Perform a new action and add it to the Undo Stack.
- **undo()** – Undo the most recent action and move it to the Redo Stack.
- **redo()** – Redo the most recently undone action and move it back to the Undo Stack.
- **displayHistory()** – Display the available Undo and Redo actions.

Example

Perform the following operations:

- performAction("Type A")
- performAction("Type B")
- performAction("Type C")
- undo()
- undo()
- redo()

Output:

- Action performed: Type A
 - Action performed: Type B
 - Action performed: Type C
 - Undo: Type C
 - Undo: Type B
 - Redo: Type B
-